

Project Documentation - UCAB: Full-Stack Cab Booking & Ride-Hailing Platform

1. Introduction

Project Title: UCAB - Full-Stack Cab Booking & Ride-Hailing Platform

Project Contributors: Niharika Devi Guttula, Madiki Devi, Mahipala Veera Venkata Manikanta, Naveen Kumar Kondepudi, and Konatham Sreevanth

Niharika Devi Guttula - Role: Team Lead & Full-Stack Developer

Niharika Devi Guttula - Contribution: Architected system, designed database schemas, developed JWT authentication, and managed API integration.

Madiki Devi - Role: Frontend Developer & UI Designer

Madiki Devi - Contribution: Designed responsive React dashboards for Users/Drivers and styled the application components.

Mahipala Veera Venkata Manikanta - Role: Backend Developer & Database Engineer

Mahipala Veera Venkata Manikanta - Contribution: Implemented Express REST APIs for ride tracking and support; configured MongoDB Atlas integration.

Naveen Kumar Kondepudi - Role: Full-stack Developer and Tester

Naveen Kumar Kondepudi - Contribution: Verified API endpoints, conducted system integration tests, handled error middleware, and resolved issues.

Konatham Sreevanth - Role: Full-stack Developer and Documentation Specialist

Konatham Sreevanth - Contribution: Contributed to full-stack development, integrated GPS Geolocation APIs, implemented persistent database backups, and compiled documentation.

2. Project Overview

Purpose: To automate and optimize the cab booking and passenger-driver matching process. It provides central coordination, geolocation resolves, street OSRM route pathing, automated fare calculations, and interactive driver availability tracking.

Features: Dual-map rendering layer (Google Maps & Leaflet maps fallbacks); dynamic geocoding (Nominatim API); draggable destination pins; simulated UPI/card/cash payment receipt gateways; forgot password database recovery resets; NH-16 3D traffic simulation.

3. Architecture

Frontend: React (Vite SPA) featuring CSS custom variables styles, Axios request filters, Google Maps SDK hooks, Leaflet interactive mapping, and central Auth Context variables.

Backend: Node.js with Express APIs using JWT session authentication, route guard parameters, and automatic persistent JSON DB backup serialization handlers.

Database: MongoDB Atlas cloud storage using Mongoose ODM schemas for User, Driver, Ride, Payment, and Support ticket models.

4. Setup Instructions

Prerequisites: Node.js, active internet connection, and any modern web browser (e.g. Chrome, Firefox, Edge).

Installation / Configuration: Clone repo; execute 'npm run install-all' to configure frontend/backend modules; populate env variables (MONGO_URI, JWT_SECRET, VITE_API_URL); run 'npm run dev' to concurrently launch server and SPA portal.

5. Application Components

User Interface: Responsive dashboards for passengers, drivers, and administrator verified approvals. Includes floating HUD overlays and billing checkout nodes.

Data Layer: Cloud MongoDB database instances synchronized with persistent local JSON backups to maintain account registration through server restarts.

Automation Layer: State-machine dispatch updates matching passengers, starting navigation loops, and printing transaction bills.

Security Layer: JWT authentication filters, bcryptjs passwords hashing, and role-based Express endpoint verification.

6. Running the Application

Onboarding / User Flow: Log in as passenger, permit GPS geolocating, search destinations dynamically, request cab matching, track curves routing, pay via simulated UPI QR code, print receipts.

Driver Flow: Log in as driver, toggle online shifts, view incoming ride cards, accept, navigate street segments, mark arrivals, complete rides.

Admin Flow: Log in as admin, approve driver onboarding requests, manage customer support tickets, review global booking parameters.

7. Workflow Operations

Key Operations: Bilingual locale swaps (English & Telugu), geolocation reverse-geocodes, OSRM road curve routing pathing, draggable destination markers, auto-checkout countdowns.

8. Authentication and Access

Security: JWT authorizations verify user sessions. Role guards protect passenger-exclusive booking desks, driver-exclusive trip controls, and admin-exclusive verification dashboards.

9. User Interface

Interface: Modern glassmorphic floating HUD cards on mapping containers, animated pulsing matchmaking radar, Neon flowing route paths, and smooth vehicle markers glide transitions.

10. Testing

Tests Performed: Verified dynamic geocoding queries under Nominatim; tested route rendering polyline outputs; simulated end-to-end driver matching queue timers; tested mobile responsive stacking configurations.

11. Demo Link

https://drive.google.com/file/d/1EUKk-PCnRqCHPiMmYS2czJ8_a0bRNhfy/view?usp=sharing

12. Known Issues

Limitations: None. The application utilizes a live MongoDB Atlas cloud database, persistent JSON backup synchronization, real-time GPS geolocation API resolution, and OSRM curves routing.

13. Future Enhancements

Enhancements: Real-time WebSocket coordinates updates stream between driver and passenger portals; live commercial Stripe payment gateways; native mobile wrapping via React Native.